lab GCA session 4 Sample Report

thomasfeys10

February 2025

1 Part 1 Coalesced memory access

Coalesced memory access happens when the threads in a warp access consecutive memory locations. This allows the memory access to be grouped and executed in parallel. Alternatively, if each thread in a warp accesses a non-consecutive memory location, each memory access happens separately.

To illustrate this, we invert the red pixels in an image as $r = r - 255$. We do this for two different image configurations, one where the pixels are ordered in $rgb, rgb, rgb, \cdots$ and one where they are ordered as $rr\cdots, gg\cdots, bb\cdots$.

When operating on the $rgb, rgb, rgb, \cdots$ image each red pixel is in a non-consecutive memory location hence, each thread in a warp will access a non-consecutvie memory location so memory access will not be coalesced.

Alternatively, when working with the $rr\cdots, gg\cdots, bb\cdots$ image, all read pixels occupy consecutive memory locations; hence, memory access will be coalesced.

The two kernels are given bellow:

```
__global__ void invert_gpu_coal(uint8_t* image_array_in){
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < M * N ) { // only red pixels
        image_array_in[idx] = 255 - image_array_in[idx];
    }
}

__global__ void invert_gpu_uncoal(uint8_t* image_array_in){ //todo
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idx_red = 3 * idx; // only red pixels
    if (idx_red < M * N * C) {
        image_array_in[idx_red] = 255 - image_array_in[idx_red];
    }
}
```

The timing results can be seen in Figs. 1 2 and 3. As can be noted the difference is only visible for very large image sizes, where the memory transfer dominates the execution time.

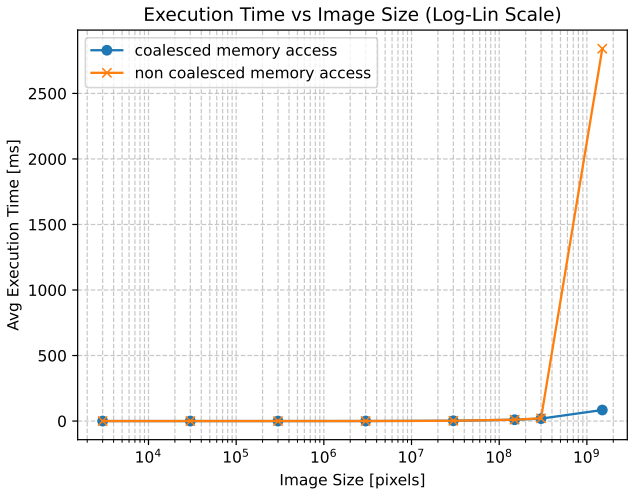


Figure 1: Avg execution times vs img size

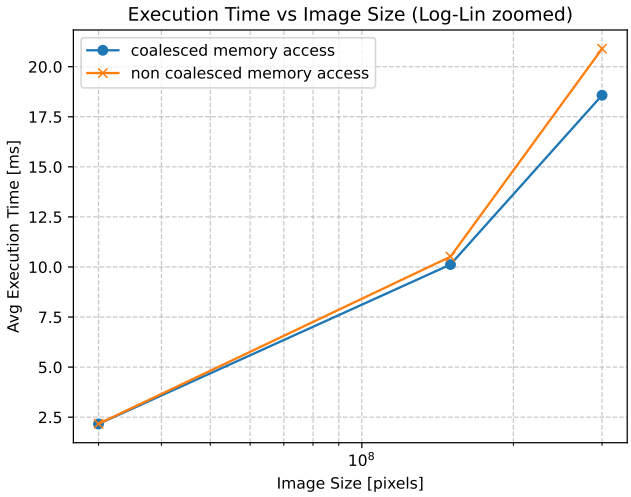


Figure 2: Avg execution times vs img size

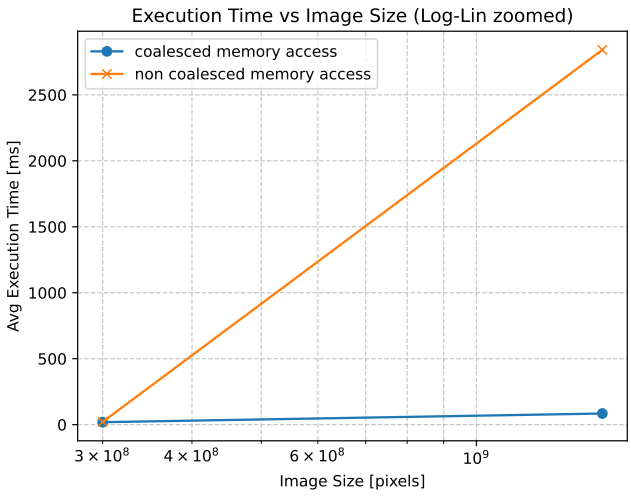


Figure 3: Avg execution times vs img size

2 Part 2 Matrix Multiply with global, shared and constant memory

2.1 Global memory

For simplicity, here we focus on square matrices to multiply, this gives $\mathbf{C} = \mathbf{AB}$. Note that the number of columns and rows is given by N_c and N_r respectivelly. We start by generating two matrices using nested vectors:

```
// Function to generate a matrix filled with 1, 2, 3, ..., n
vector<vector<int>> generateMatrix(int rows, int cols) {
    vector<vector<int>> matrix(rows, vector<int>(cols));
    int value = 1;

    // Fill the matrix with sequential values
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = value++;
        }
    }

    return matrix;
}

 // Generate matrix
 vector<vector<int>> A = generateMatrix(rows, cols);
 // Generate matrix
 vector<vector<int>> B = generateMatrix(rows, cols);
```

However, CUDA works best on 1D arrays, hence we will flatten (and unflatten) the matrices using the following functions

```
// Function to flatten a 2D vector into a 1D vector
vector<int> flattenMatrix(const vector<vector<int>>& matrix) {
    int rows = matrix.size();
    int cols = matrix[0].size();
    vector<int> flattened(rows * cols);

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            flattened[i * cols + j] = matrix[i][j];
        }
    }

    return flattened;
}

// Function to unflatten a 1D vector into a 2D vector
vector<vector<int>> unflattenMatrix(const vector<int>& flattened, int rows, int cols) {
    vector<vector<int>> matrix(rows, vector<int>(cols));

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = flattened[i * cols + j];
        }
    }

    return matrix;
}
```

Then we can call the kernel as

```
// Flatten matrices for CUDA (more efficient in 1D form)
vector<int> A_flat = flattenMatrix(A);
vector<int> B_flat = flattenMatrix(B);

// Allocate memory on the device
int *d_A, *d_B, *d_C;
size_t sizeA = rows * cols * sizeof(int);
size_t sizeB = rows * cols * sizeof(int);
size_t sizeC = rows * cols * sizeof(int);

cudaMalloc(&d_A, sizeA);
cudaMalloc(&d_B, sizeB);
cudaMalloc(&d_C, sizeC);

// Copy data to device
cudaMemcpy(d_A, A_flat.data(), sizeA, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, B_flat.data(), sizeB, cudaMemcpyHostToDevice);

// Define block and grid size
dim3 blockSize(16, 16);  // 16x16 threads per block
dim3 gridSize((cols + blockSize.x - 1) / blockSize.x, (rows + blockSize.y - 1) / blockSize.y);

// call kernel
squareMatrixMultiplyKernel<<<gridSize, blockSize>>>(d_A, d_B, d_C, rows, cols);

// Copy the result back to the host
vector<int> C_flat(rows * cols);
cudaMemcpy(C_flat.data(), d_C, sizeC, cudaMemcpyDeviceToHost);

// Get C back in matrix form
vector<vector<int>> C_gpu = unflattenMatrix(C_flat, rows, cols);

// Print matrix
cout << "C GPU: " << "\n" << C_gpu;

// Free device memory
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

The matrix multiplication kernel is given by

```
__global__ void squareMatrixMultiplyKernel(int* A, int* B, int* C, int num_rows, int num_cols) {
    int row_idx = threadIdx.x + blockIdx.x * blockDim.x;
    int col_idx = threadIdx.y + blockIdx.y * blockDim.y;
    int one_d_idx = row_idx * num_cols + col_idx;
    if (row_idx < num_rows && col_idx < num_cols){ // only operate within the matrix
        for(int i=0; i<num_cols; i++){
            int one_d_idx_A = i*num_cols + col_idx;
            // in 2D goes from (row_idx, 0) to (row_idx, num_col)
            int one_d_idx_B = row_idx*num_cols + i ;
            // in 2D goes from (0, col_idx) to (num_rows, col_idx)
            C[one_d_idx] += A[one_d_idx_A] * B[one_d_idx_B];
        }
    }
}
```

Note that the conversion between the 2D index (i, j) (row i, column j) to the 1D index of the flattened matrix is $idx_{1D} = i \cdot N_c + j$, where N_c is the number of columns in the matrix. Then we note that each thread computes one value of the output array namely the thread with row index i and columns index j computes $[\mathbf{C}]_{i,j}$. For this

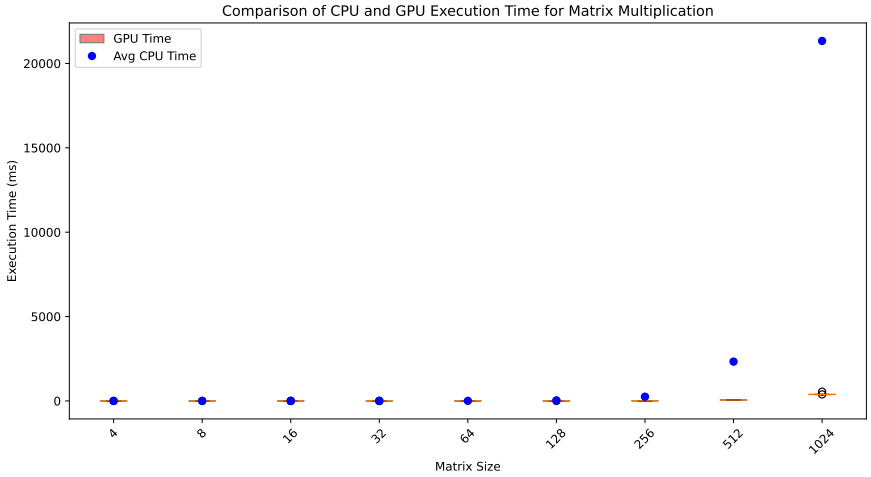


Figure 4: Execution time: matmul on CPU and GPU vs matrix size

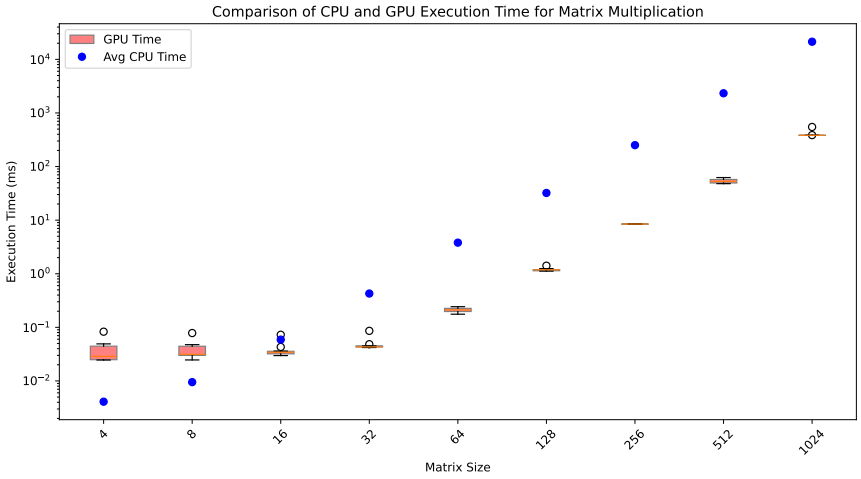


Figure 5: Execution time: matmul on CPU and GPU vs matrix size log scale

we loop over the size of the matrix (= number of columns = number of rows cause square matrix). Then based on this loop we need to compute the correct index for matrix $\mathbf{A}$ and $\mathbf{B}$. We note that matrix $\mathbf{A}$ is indexed from $(i, 0)$ up to (i, N_c) and matrix $\mathbf{B}$ is indexed from $(0, j)$ up to (N_r, j). In order to achieve this we have the following indexing scheme:

```
for(int i=0; i<num_cols; i++){
            int one_d_idx_A = i*num_cols + col_idx;
            // in 2D goes from (row_idx, 0) to (row_idx, num_col)
            int one_d_idx_B = row_idx*num_cols + i ;
            // in 2D goes from (0, col_idx) to (num_rows, col_idx)
            C[one_d_idx] += A[one_d_idx_A] * B[one_d_idx_B];
        }
```

Timing this code we get Figs. 4 5, note that this excludes memory transfers!.

For completeness, we also include the timing including memory transfer in Fig. 6, we see that we have a gain here from size 32 instead of 16 when excluding memory transfer.

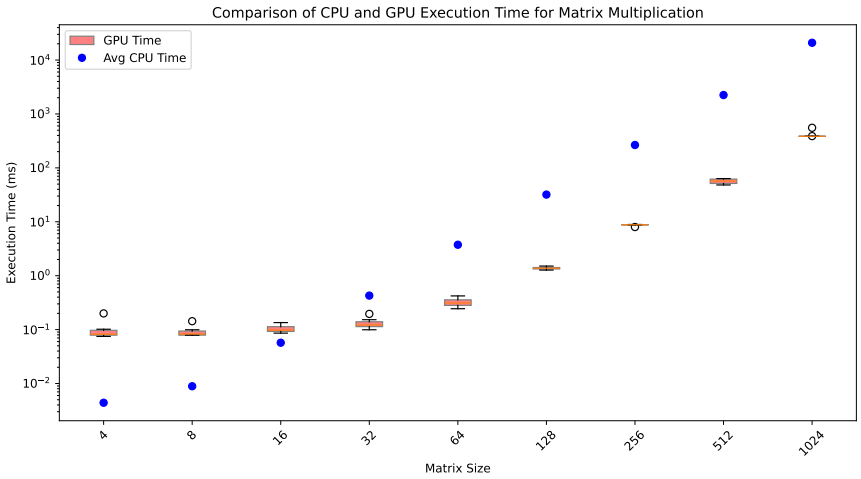


Figure 6: Execution time: matmul on CPU and GPU vs matrix size with memory transfer.